

HAKS 2026 — Demo-Day Runbook

IBM Bob × Airbus × AWS · Aircraft-maintenance AI

Project **JacquesLePluBo** — one-day hackathon · 5h build (14:00–19:00) · pitch 19:00

What this is: three Airbus maintenance use cases, already wired end-to-end against realistic synthetic data, so whichever brief Airbus reveals at 13:00 you start from a running demo — not a blank repo. Two of the three run with **zero AWS dependency**; the third (RAG) has an offline fallback so it demos even if Bedrock isn't configured.

Judging criteria (optimise in order): quantified business value · a live demo that actually runs · scalability & data sovereignty.

Use case	Runs offline?	Headline metric (synthetic)
#1 RAG doc assistant	Yes (TF-IDF); Bedrock optional	Cited answers, source traceability
#2 NLP maintenance triage	Yes (spaCy + rules)	100% ATA classification, dedup, priority queue
#3 Predictive maintenance	Yes (scikit-learn)	Recall 0.55–0.92 vs labelled anomalies; RUL est.

1 · Setup — get running in ~10 minutes

Each teammate, on their own laptop (Linux/macOS). The **venv is not in git** — create it.

```
git clone git@github.com:TABguy/JacquesLePluBo.git
cd JacquesLePluBo
python3 -m venv venv && source venv/bin/activate
pip install --upgrade pip && pip install -r requirements.txt
python -m spacy download fr_core_news_lg # French NER model (~560 MB)
cp .env.example .env # fill in Bedrock IDs (only #1 needs them)
```

Generate data + launch (all offline)

```
python src/gen_data.py # maintenance logs + work-order/procedure PDFs -> input/
python src/gen_sensors.py # labelled sensor time series -> input/sensors/
python src/ingest.py --offline # build the TF-IDF index for RAG (no AWS)
streamlit run app.py # the 3-tab demo dashboard
```

- **Verify AWS (only for #1's Bedrock path):** `python src/common.py` — if it prints a model count, Bedrock is reachable in eu-west-1.
- Data is **deterministic** (seeded) — every laptop produces identical data.
- **Never commit** `.env` or AWS keys.

Repo map

File	Role
<code>src/gen_data.py</code>	Synthetic maintenance logs (CSV/JSONL) + work-order/procedure PDFs
<code>src/gen_sensors.py</code>	Synthetic sensor series with injected, labelled anomalies (#3)
<code>src/common.py</code>	Config (<code>.env</code>) + Bedrock client + connectivity check
<code>src/ingest.py</code>	#1 build index: TF-IDF (offline) or Bedrock→Chroma (<code>--offline</code> flag)
<code>src/rag.py</code>	#1 retrieval + cited answer; auto-picks Bedrock or offline backend
<code>src/nlp_triage.py</code>	#2 spaCy NER + rules + optional Bedrock; dedup, priority, summarise
<code>src/anomaly.py</code>	#3 IsolationForest + fleet health + trend/RUL + metrics
<code>app.py</code>	Streamlit UI — one tab per use case

2 · The three use cases (what to run, how to adapt)

#1 · RAG assistant over technical documentation — recommended priority

Story: a mechanic asks in natural language and gets the right procedure with its source reference / ATA chapter. Immobilisation time = money; traceability sells to the jury.

Run: `python src/ingest.py --offline` then ask in the RAG tab (works with no AWS). With `BEDROCK_MODEL_ID+EMBED_MODEL_ID` set, `python src/ingest.py` builds a Bedrock/Chroma index and answers become LLM-generated.

Adapt to the real brief: drop the real (or better synthetic) PDFs into `input/` and re-run `ingest` — the loader already reads any PDF. Tune chunk size / top-k in `ingest.py` / `rag.py`. For the IBM angle, the optional `src/ingest_docling.py` swaps in Docling layout-aware parsing (heavy — see §5).

#2 · NLP triage of maintenance reports / logbooks

Story: noisy free-text PIREP/logbook entries → auto-classified by ATA chapter, entities extracted, duplicates merged, prioritised by criticality. Two-stage (deterministic rules first, LLM enrichment second) signals reliability, not just 'all-LLM'.

Run (offline): Triage tab → 'Triage full corpus'. Stage 1 hits **100% ATA accuracy** on the synthetic set at zero LLM cost; Bedrock Stage 2 is optional enrichment.

Adapt: edit the `ATA_KB` dict and priority weights in `nlp_triage.py` to match the brief's taxonomy; point `triage_file()` at the real input. Outputs a prioritised `input/triaged.csv`.

#3 · Predictive maintenance / anomaly detection

Story: sensor series → anomalies flagged, component health scored, rough remaining-useful-life (RUL) estimated, fleet ranked worst-first. High storytelling (safety + cost) but **data-risky** — which is why the generator injects labelled anomalies so the demo shows real precision/recall.

Run (offline): `python src/gen_sensors.py` then the Predictive tab — fleet table + health bars + per-sensor anomaly plot.

Adapt: add sensors / change baselines, degradation and failure thresholds in `gen_sensors.py`; swap `IsolationForest` or tune `contamination` in `anomaly.py`. If the brief brings real sensor data, point `fleet_summary()` at it.

3 · At 13:00 — pick ONE, then focus

All three are wired, but a hackathon is won by one tight demo. Decide fast, then **delete the other two tabs/modules** so the repo and the pitch stay focused.

If the brief is about...	Go with	Why
Finding procedures in manuals (AMM/IPC/SB), Q&A over docs	#1 RAG	Spectacular demo, clear traceability, the safe pick
Sorting/structuring free-text reports, logbooks, PIREPs, ATA tagging	#2 Triage	Two-stage reliability story lands with an industrial jury
Sensor health, failures, Skywise-like monitoring	#3 Predictive	Strong story — only if data holds up (it does, it's labelled)

4 · Demo script (≈3 minutes)

- 1 **Frame the pain (20s):** a mechanic/controller loses time on X; downtime costs money.
- 2 **Show the data (15s):** realistic synthetic ATA-coded data in input / — no real Airbus data needed.
- 3 **Run it live (90s):** the chosen tab — ask a question / triage the corpus / open the fleet view. Point at the **citation / ATA accuracy / recall** on screen.
- 4 **Quantify (30s):** minutes saved × volume/year (see §6).
- 5 **Sovereignty (15s):** runs on AWS eu-west-1; offline modes prove no data leaves the environment.

5 · Business value — fill the real numbers in the pitch

Template per use case (replace the illustrative figures with the brief's context):

Use case	Lever	Illustrative calc (replace!)
#1 RAG	Minutes saved per procedure lookup × lookups/year	5 min × 50k lookups = ~4,150 h/yr
#2 Triage	Manual sorting time saved + faster fault detection	3 min × 200k reports = ~10,000 h/yr
#3 Predictive	Unplanned AOG events avoided × cost per event	10 events × €150k = €1.5M/yr

Scalability & sovereignty talking points: local vector store / offline modes keep data in-region; Bedrock in eu-west-1; no fine-tuning, no data leaving the account; the two-stage triage shows deterministic guardrails around the LLM.

6 · Heavy optional extra — Docling (IBM angle)

`src/ingest_docling.py` uses Docling for layout-aware PDF parsing. It is **deliberately not** in the base env: it pulls torch + the CUDA toolkit (several GB) and forces numpy 2.x, which conflicts with the pinned stack. Install only on a capable machine, in time before the demo:

```
pip install -r requirements-docling.txt # heavy: torch + CUDA
```

The default `src/ingest.py` needs none of it.

7 · Troubleshooting

Symptom	Fix
streamlit: command not found	source venv/bin/activate
spaCy model missing	python -m spacy download fr_core_news_lg
RAG: 'run ingest first' / empty	python src/ingest.py --offline
Bedrock AccessDenied / region	check .env AWS_REGION=eu-west-1 + model access enabled
pip resolution-too-deep	keep the upper bounds in requirements.txt; don't add llama-index meta
No sensor data in #3	python src/gen_sensors.py

Freeze code at 18:15 — last 45 min for demo + pitch rehearsal. Park new ideas in BACKLOG.md, don't build them. Generated by `src/make_runbook.py`.